

JavaScript Functions — Basic to Advanced

A **function** is a reusable block of code designed to perform a specific task.

Real-life example

Think of a **washing machine button**:

- You press the button → machine performs a task.
 - You don't need to write the washing process every time.
 - Similarly, a JavaScript function can be called whenever you need that task.
-

1. Why Do We Need Functions?

Without functions:

```
console.log("Welcome Vidya");  
console.log("Welcome Anurag");  
console.log("Welcome Pooja");
```

With a function:

```
function welcome() {  
  console.log("Welcome to JavaScript");  
}
```

```
welcome();  
welcome();  
welcome();
```

Advantages

- Code reusability
- Less code duplication
- Easy maintenance
- Better organization
- Easier debugging

2. Basic Function Syntax

```
function functionName() {  
    // statements  
}
```

Example:

```
function greet() {  
    console.log("Hello Students");  
}
```

But this function **doesn't execute yet**.

To execute it, we need to **call/invoke** the function:

```
greet();
```

Output:

Hello Students

3. Function Declaration

The traditional way of creating a function is called a **function declaration**.

```
function add() {  
    console.log(10 + 20);  
}
```

```
add();
```

Output:

30

4. Function Calling

Creating a function:

```
function sayHello() {  
    console.log("Hello");  
}
```

Calling a function:

```
sayHello();
```

You can call it multiple times:

```
sayHello();  
sayHello();  
sayHello();
```

Output:

```
Hello  
Hello  
Hello
```

5. Function Parameters

A **parameter** is a variable that receives a value when the function is called.

```
function greet(name) {  
  console.log("Hello " + name);  
}
```

```
greet("Ravi");  
greet("Anu");
```

Output:

```
Hello Ravi  
Hello Anu
```

Here:

```
name
```

is the **parameter**.

And:

```
"Ravi"  
"Anu"
```

are **arguments**.

Important

Parameter → variable defined in function

Argument → actual value passed to function

6. Multiple Parameters

```
function add(a, b) {  
  console.log(a + b);  
}
```

```
add(10, 20);
```

Output:

30

Another example:

```
function student(name, age, course) {  
  console.log(name);  
  console.log(age);  
  console.log(course);  
}
```

```
student("Ravi", 21, "Python");
```

Output:

Ravi
21
Python

7. Return Statement

Sometimes we don't want the function to print the result.

We want the function to **send the result back**.

For this, we use **return**.

```
function add(a, b) {  
  return a + b;  
}
```

```
let result = add(10, 20);
```

```
console.log(result);
```

Output:

30

Difference

Using `console.log()`:

```
function add(a, b) {  
  console.log(a + b);  
}
```

Using `return`:

```
function add(a, b) {  
  return a + b;  
}
```

`return` is more useful because we can store and reuse the result.

```
let result = add(10, 20);
```

```
console.log(result);  
console.log(result * 2);
```

Output:

30
60

8. Function Stops After Return

```
function test() {  
  return 10;  
  
  console.log("Hello");  
}
```

```
console.log(test());
```

Output:

10

The statement after `return` doesn't execute.

9. Function With No Parameter and No Return

```
function message() {  
  console.log("Welcome");  
}
```

```
message();
```

10. Function With Parameters but No Return

```
function add(a, b) {  
  console.log(a + b);  
}
```

```
add(10, 20);
```

11. Function With Parameters and Return

```
function add(a, b) {  
  return a + b;  
}
```

```
let x = add(10, 20);
```

```
console.log(x);
```

This is one of the most commonly used patterns.

12. Function With No Parameter but Return

```
function getNumber() {
```

```
    return 100;
}

let number = getNumber();

console.log(number);
```

Output:

100

13. Default Parameters

You can provide a default value for a parameter.

```
function greet(name = "Student") {
    console.log("Hello " + name);
}
```

```
greet("Ravi");
greet();
```

Output:

Hello Ravi
Hello Student

Another example:

```
function calculate(a, b = 10) {
    return a + b;
}
```

```
console.log(calculate(20));
```

Output:

30

14. Function Expression

A function can be stored inside a variable.

```
let greet = function() {
```

```
    console.log("Hello");  
};
```

```
greet();
```

Here:

```
let greet
```

stores the function.

15. Function Expression With Parameters

```
let add = function(a, b) {  
    return a + b;  
};
```

```
console.log(add(10, 20));
```

Output:

30

16. Anonymous Function

A function without a name is called an **anonymous function**.

```
let message = function() {  
    console.log("Hello");  
};
```

```
message();
```

The function itself has no name.

17. Arrow Functions

Modern JavaScript provides **arrow functions**.

Traditional:

```
function add(a, b) {  
  return a + b;  
}
```

Arrow function:

```
const add = (a, b) => {  
  return a + b;  
};
```

Call:

```
console.log(add(10, 20));
```

18. Arrow Function — Short Syntax

If the function contains only one return statement:

```
const add = (a, b) => a + b;
```

```
console.log(add(10, 20));
```

Output:

30

This is called an **implicit return**.

19. One Parameter Arrow Function

```
const square = n => n * n;
```

```
console.log(square(5));
```

Output:

25

Parentheses are optional for one parameter:

```
n => n * n
```

But for multiple parameters:

```
(a, b) => a + b
```

20. Zero Parameter Arrow Function

```
const greet = () => {  
  console.log("Hello Students");  
};
```

```
greet();
```

21. Function Scope

Variables created inside a function normally belong to that function.

```
function test() {  
  let x = 100;  
  
  console.log(x);  
}
```

```
test();
```

This works.

But:

```
function test() {  
  let x = 100;  
}
```

```
console.log(x);
```

This produces an error because `x` is local to the function.

22. Local Variable

```
function student() {  
  let name = "Ravi";  
  
  console.log(name);  
}  
  
student();  
  
name is a local variable.
```

23. Global Variable

```
let college = "ABC College";  
  
function display() {  
  console.log(college);  
}  
  
display();  
  
The function can access the global variable.
```

24. Nested Functions

A function can be created inside another function.

```
function outer() {  
  
  function inner() {  
    console.log("Inner function");  
  }  
  
  inner();  
}  
  
outer();  
  
Output:
```

Inner function

25. Function Calling Another Function

```
function add(a, b) {  
  return a + b;  
}  
  
function display() {  
  let result = add(10, 20);  
  console.log(result);  
}
```

display();

Output:

30

26. Callback Functions

One of the most important JavaScript concepts.

A function can be passed as an argument to another function.

```
function greet() {  
  console.log("Hello");  
}  
  
function execute(callback) {  
  callback();  
}
```

execute(greet);

Output:

Hello

Here:

greet

is passed as a callback.

27. Callback With Parameters

```
function add(a, b) {  
  return a + b;  
}
```

```
function calculate(callback) {  
  console.log(callback(10, 20));  
}
```

```
calculate(add);
```

Output:

30

28. Anonymous Callback

```
function calculate(callback) {  
  console.log(callback(10, 20));  
}
```

```
calculate(function(a, b) {  
  return a + b;  
});
```

29. Arrow Callback

```
function calculate(callback) {  
  console.log(callback(10, 20));  
}
```

```
calculate((a, b) => a + b);
```

30. Higher-Order Functions

A function that:

- accepts another function as an argument, **or**
- returns another function

is called a **higher-order function**.

Example:

```
function calculate(operation) {  
  return operation(10, 20);  
}
```

```
let result = calculate((a, b) => a + b);
```

```
console.log(result);
```

Output:

30

31. Function Returning Another Function

```
function outer() {  
  
  return function() {  
    console.log("Hello");  
  };  
}
```

```
let result = outer();
```

```
result();
```

Output:

Hello

32. Rest Parameters

Rest parameters allow a function to accept many arguments.

```
function add(...numbers) {  
  console.log(numbers);  
}
```

```
add(10, 20, 30, 40);
```

Output:

```
[10, 20, 30, 40]
```

Example:

```
function add(...numbers) {  
  
  let total = 0;  
  
  for (let number of numbers) {  
    total += number;  
  }  
  
  return total;  
}
```

```
console.log(add(10, 20));  
console.log(add(10, 20, 30));  
console.log(add(10, 20, 30, 40));
```

Output:

```
30  
60  
100
```

33. Spread vs Rest

Rest:

```
function add(...numbers) {  
}
```

Collects multiple values.

Spread:

```
let numbers = [10, 20, 30];
```

```
console.log(...numbers);
```

Expands values.

Remember

Rest → Collect

Spread → Expand

34. Recursive Function

A function calling itself is called **recursion**.

Example:

```
function count(n) {  
    if (n === 0) {  
        return;  
    }  
  
    console.log(n);  
  
    count(n - 1);  
}
```

```
count(5);
```

Output:

```
5  
4  
3  
2  
1
```